Algoritm nedeterminst -> un concept pur teoretic
-> util pentru a clasifica algoritmii in diverse clase de complexitate

3 operatii speciale:
choice()
fail()
success()

-> Un singur thread intoare success -> tot algoritmul se termina cu succes

x = choice(1..n)
x = 1 x=2 x=3 x=4 .. x= n
// n threaduri
y = choice (1.. n)
x=1,y=1 x=1,y=2 x=1,y=3.. x=1,y=n .. x=n,y=1.. x=n,y=n
// n^2 threaduri

```
for i=1:n
        V[i]= choice(1..n)
```

n^n threaduri -> toate posibilitatile da a pune numerele de la 1 la n in vector

{1..n}x{1..n}x{1..n}x ..x{1..n} -> fiecare posibilitate pe propriul thread

permutari:

```
V[n] = {0}
marcat[n] = {0}
for i=1:n
        V[i] = choice(1..n)
        if marcat[V[i]] == true:
                fail()
        marcat[V[i]] = true
```

n! threaduri

aranjamente:

```
V[k] = {0}
marcat[n] = {0}
for i =1:k
        V[i] = choice(1..n)
        if marcat[V[i]] == true:
                fail()
        marcat[V[i]] = true
```

(1,2,3) (2,3,1) (1,2,4) (2,4,1) ..
-> complexitate "angelica" Theta(n) pentru permutari, Theta(k) pentru aranjamente

-> generare
-> testare: daca se verifica o singura multime generata, algoritmul intoarce succes

Clasa NP: Algoritmi care se pot rezolva in timp polinomial folosind algoritmi nedeterministi
Clasa P: Algoritmi care se pot rezolva in timp polinomial folosind algoritmi deterministi
Observatie: P este inclus sau egal in NP

Algoritmi nedetermnisti -> algoritmi de decizie, intorc true sau false
-> daca un singur thread intoarce success(), tot programul se termina cu succes

problema comis voiajorului (ciclu hamiltomian)

- sa se gaseasca un ciclu care trece prin toate nodurile o singura data
a) care sa fie mai mic decat un K dat (varianta de decizie)
b) si are cost minim (varianta de optim)
→ se poate modela folosind mai multe probleme de decizie, astfel
exista un ciclu de cost maxim 1? NU
exista un ciclu de cost maxim 2? NU
exista un ciclu de cost maxim 3? NU
…
exista un ciclu de cost maxim K? DA

```
TSP(G,n,k)
//generare
V[n] = {0}
marcat[n] = {0}
for i=1:n
        V[i] = choice(1..n)
        if marcat[V[i]] == true:
                fail()
        marcat[V[i]] = true
//  testare
cost = 0
for i=1:n
        if (V[i],V[i+1]) not in G.Edges:
                fail()
        cost += cost_muchie(V[i],V[i+1])

if (V[n], V[1]) not in G.Edges:
        fail()
cost += cost_muchie(V[n],V[1])
if cost >K:
        fail()
success()
```

Problema grafurilor izomorfe:

2 grafuri -> G1 si G2

exista cumva un subgraf in G1 care este izomorf cu graful G2?

graph_isomorphism(G1,G2,N1,N2)

```
if N1 < N2:
        fail()

V[N2] = {0}
marcat[N1] = {0}

for i =1 :N2
        V[i] = choice(1..N1)
        if marcat[V[i]] == true:
                fail()
        marcat[V[i]] = true

for i=1:N2
        for j=1:N2
                if (i,j) in G2.Edges and (V[i],V[j]) not in G1.Edges:
                        fail()
                if (i,j) not in G2.Edges and (V[i],V[j]) in G1.Edges:
                        fail()
succes()

find(A,M,N,k)
for i=1:M
        for j=1:N
                if (A[i][j] == k)
                        return 1
return 0

find2(A,M,N,k)
i = choice(1..M)
for j=1:N
        if (A[i][j] ==k)
                success()
fail()

find3(A,M,N,k)
i = choice(1..M)
j = choice(1..N)
if (A[i][j] == k)
        success()
fail()
```

reduceri:

A <=p B

A este "cel mult la fel de grea" ca B

1. Exista o functie F care ia orice input a lui A si il transforma intr-un input F(i) pentru problema B

2. A(i) =1 <=> B(F(i)) = 1 (trebuie aratat si de la stanga la dreapta, si invers)

NP-Hard: A este NP-Hard daca oricare B din NP, B <=p A
NPC: A este NPC daca A este NP si A este NP-Hard

Problema SAT este NP-Hard (chiar NPC)
-> problema satisfiabilitatii expresiilor booleene

Orice problema Q din NP are proprietatea ca Q<=p SAT
Daca arat ca SAT <=p X => Q<=p SAT <=p X => X este NP-Hard

-> Ca sa aratam ca o problema P este NP-Hard (NPC) iau o problema despre care stiu deja ca este NP-Hard si o reduc la problema mea P.

Partitie: Se da o multime A de N numere. Poate fi A partitionata in 2 submultimi de suma egala?
Q-Sume: Se da o multime B de M numere si un numar q. Putem gasi o submultime de numere din B cu suma Q?

Partitie <=p Q-Sume

Functia de transformare: B=A, M=N. Q = sum(A)/2
Daca pot partitiona multimea A in 2 submultimi de suma egala => ele vor avea suma = Sum(A)/2 deci in Q-sume voi gasi o submultime egala cu sum(A)/2

Vertex cover <= Independent set

Vertex cover: pentru un graf G si un numar k, se poate alege o submultime S de k noduri a.i. oricare muchie are cel putin un capat in S?
Independent set: pentru un graf G si un numar k, se poate alege o submultime S de k noduri a.i. oricare muchie are cel mult un capat in S?

Vertex cover de dimensiune k => orice muchie (u,v) are cel putin un capat in submultimea mea S => daca ma uit la submultimea V-S, orice muchie (u,v) va avea cel mult un capat in submultimea V-S => V-S este un independent set => k' = n-k pentru independent set

Vertex_cover <= Q sume

-> se codifica fiecare nod si fiecare muchie din graf, ele vor forma numerele din submultimea S

		m1	m2	m3..	me
n1	1	1	0	0..	1
n2	1	0	1	0	0
n3	1	1	0	1	0
n4	1	0	1	1	0
nV	1				0

m1 -> muchie de la n1 la n3 => (n1,m1) =1 si (n1,m3) = 1

m1: 0 1 0 0 0 .. 0
m2: 0 0 1 0 0 .. 0
m3: 0 0 0 1 0 ..
...
me: 0 0 .. 1

S va contine toate codificarile pentru noduri si pentru muchii

S= {n1,n2,n3...nV,m1,m2,m3...me}

Daca exista un vertex cover de dimensiune k:

$q = k * 4^{|E|} + \sum_{j=0}^{|E|-1} 4^j * 2$

Pentru o muchie mi = (nu, nj) din vertex cover, daca muchia are ambele capete in vertex cover atunci in q-sume voi alege doar codificarile pentru nodurile nu si nj, altfel aleg, in plus, si codificarea pentru muchia mi, a.i. pe bitul i sa am mereu suma 2
Bitul cel mai semnificativ este k => Am nevoie neaparat de k codificari pentru noduri
=> se aleg acele noduri care sunt in vertex cover plus muchiile care au doar un capat incident in submultimea mea

P1:
daca Q1 apartine P si Q2 <=p Q1 => Q2 apartine P.

P2:
daca Q1 <=p Q2 si Q2 <=p Q3 => Q1 <=p Q3.

Teoreme:

T1:
Q2 apartine NPD <=> exista Q1 din NPC, Q1 <=p Q2.

=> Q2 este NPD => orice Q din NP, Q<=p Q2, dar Q1 este din NPC deci si np =>
Q1 <=p Q2

<= Q1 este din NPC => Q1 este din NPD, orice Q din NP, Q<=p Q1, Q1 <=p Q2 =>
Q<=p Q2 => Q2 este NPD

T2:

Q1, Q2 apartin NPC => Q1 <=p Q2 si Q2 <=p Q1

Q1 este NPC deci Q1 este si NPD => oricare Q din NP, Q <=p Q1. Dar Q2 este NPC =>
Q2 este si NP => Q2 <=p Q1
analog pentru Q1 <=p Q2

T3:
daca NPC intersectat cu P != multimea vida => P =NP.

Corolar: daca NPC intersectat cu P != multimea vida => NPC este inclus in P

Fie Q care este in NPC dar este si in P.
Cum Q este NPD => Oricare Q1 din NP, Q1<=p Q, dar Q este in P =>
Q1 este in P => NP este inclus in P
Dar P este inclus in NP => P = NP

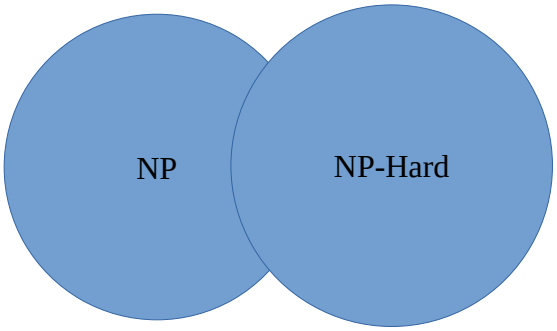
Din acest rezultat => NPC care este inclus in NP va fi inclus in P.

T4: daca NPD intersectat cu P != multimea vida => P =NP.

Corolar: daca NPD intersectat cu P != multimea vida => NPC este inclus in P

Cum Q este NPD => Oricare Q1 din NP, Q1<=p Q, dar Q este in P =>
Q1 este in P => NP este inclus in P
Dar P este inclus in NP => P = NP

Din acest rezultat => NPC care este inclus in NP va fi inclus in P.



=> intersectia NP cu NP-Hard = NPC

Exemplu: problema NP-Hard care nu este NP:
Halting problem (problema opririi): Se da un program P si o intrare x. Se opreste P(x)?
-> Halting problem nu este decidabila